DSAI LAB PROJECT · MILESTONE 3 REVIEW

Detecting AI-generated voices in Indian languages

Model architecture and a full pipeline check

Team · Group-3

5 November 2026

01 · THE GOAL

Show the detector works from start to finish

Design the model

Choose the model, show how its parts connect, and explain why we chose it.

Run every step once

Prepare a small set of clips, train for one round, then score and evaluate.

Be honest about it

Report what worked, what didn’t, and what’s still missing.

This milestone checks that every part connects. A good score comes later, with full training in Milestone 4.

02 · THE DATA

A small subset that still covers everything

Split	Real	Fake	Clips	Used for
train	Kathbath; ASVspoof train	IndicSynth; ASVspoof train	1,492	Training
valid	Kathbath; ASVspoof dev	IndicSynth; ASVspoof dev	397	Choosing the decision cut-off
test_indomain	Kathbath	IndicSynth	300	New speakers, known voice tools
test_unseen_gen	Kathbath test speakers	Our generated set	540	Voice tools never seen in training
test_itw	In-the-Wild	In-the-Wild	193	Real-world English audio
test_asvspoof	ASVspoof eval	ASVspoof eval	96	Unseen English attacks

3,018
clips, 4.73 hours

≈ 50 / 50
real / fake in every split

4
languages

25
voice tools and attack types

03 · HOW WE PICKED THE CLIPS

Fair from the start

Split by speaker

Kathbath’s training speakers are split 70 / 15 / 15, by gender, with seed 42: the split planned in Milestone 2.

Fakes follow their speaker

An IndicSynth fake is used only if every speaker it copies is in the same split.

Same words, real and fake

The unseen-tool test pairs each fake sentence with a real recording of the same sentence.

Official English splits

ASVspoof keeps its own train, dev and eval. In-the-Wild is used only for testing.

04 · HOW THE DATA IS STORED

Raw, processed, and one list

```
data/
├── raw/          the clips as downloaded
│   └── <dataset>/<language>/
├── processed/    the same clips, cleaned
│   └── <dataset>/<language>/
└── manifests/
    ├── manifest_subset.csv
    ├── speaker_split.csv
    ├── leakage_report.csv
    └── dropped.csv
```

One row per clip

The manifest lists every clip with its file, dataset, language, label, voice tool, speaker, split and hash.

Splits are a column

No separate folders: later steps read one file and pick a split.

Kept out of GitHub

1.1 GB of audio. It’s a private Kaggle dataset for the team.

05 · PREPROCESSING

One cleaning step for every clip

Decode
Mix to mono

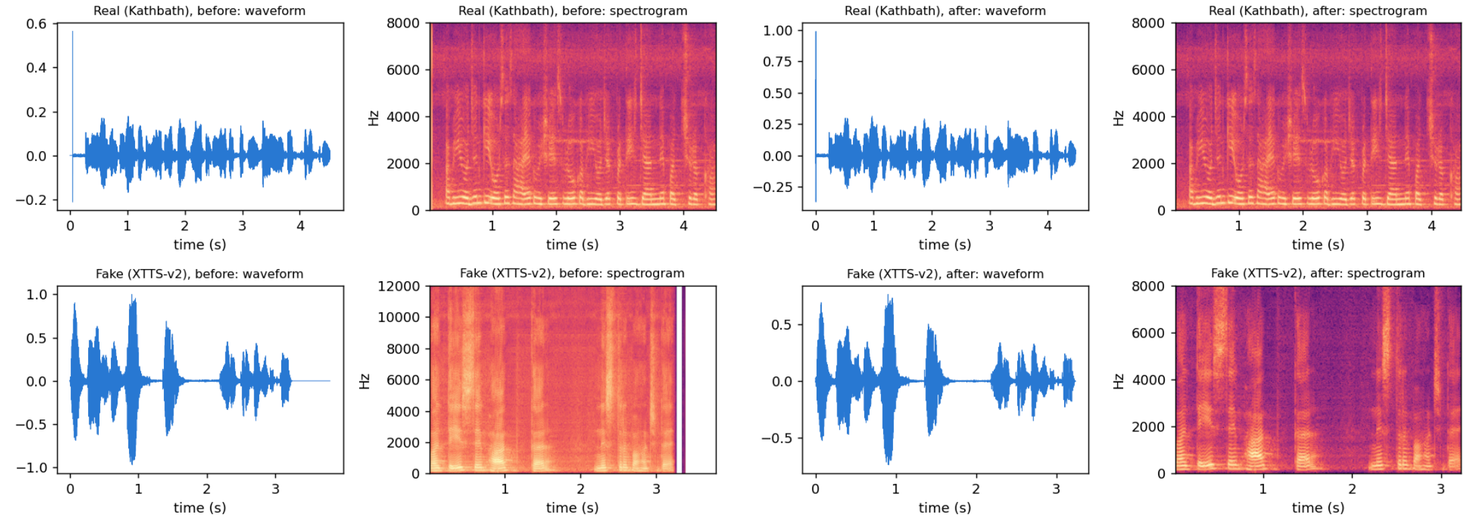
Resample
16 kHz

Trim
Silence at both ends

Drop
Under 1 s left

Normalise
−20 dBFS, peak ≤ 0.99

Store
16-bit WAV + hash



Before: the fake clip was 24 kHz and the real one 16 kHz. A model could tell them apart by that alone.

After: both are 16 kHz, with the silence at the ends cut.

22 clips had under 1 s of speech and were dropped.

06 · SPLITS AND LEAKAGE

No voice is in two places

Language	train	valid	test
Hindi	71	15	15
Tamil	96	21	21
Telugu	59	12	13

Speakers per split, balanced by gender.

0
speakers in two splits

0
duplicate clips

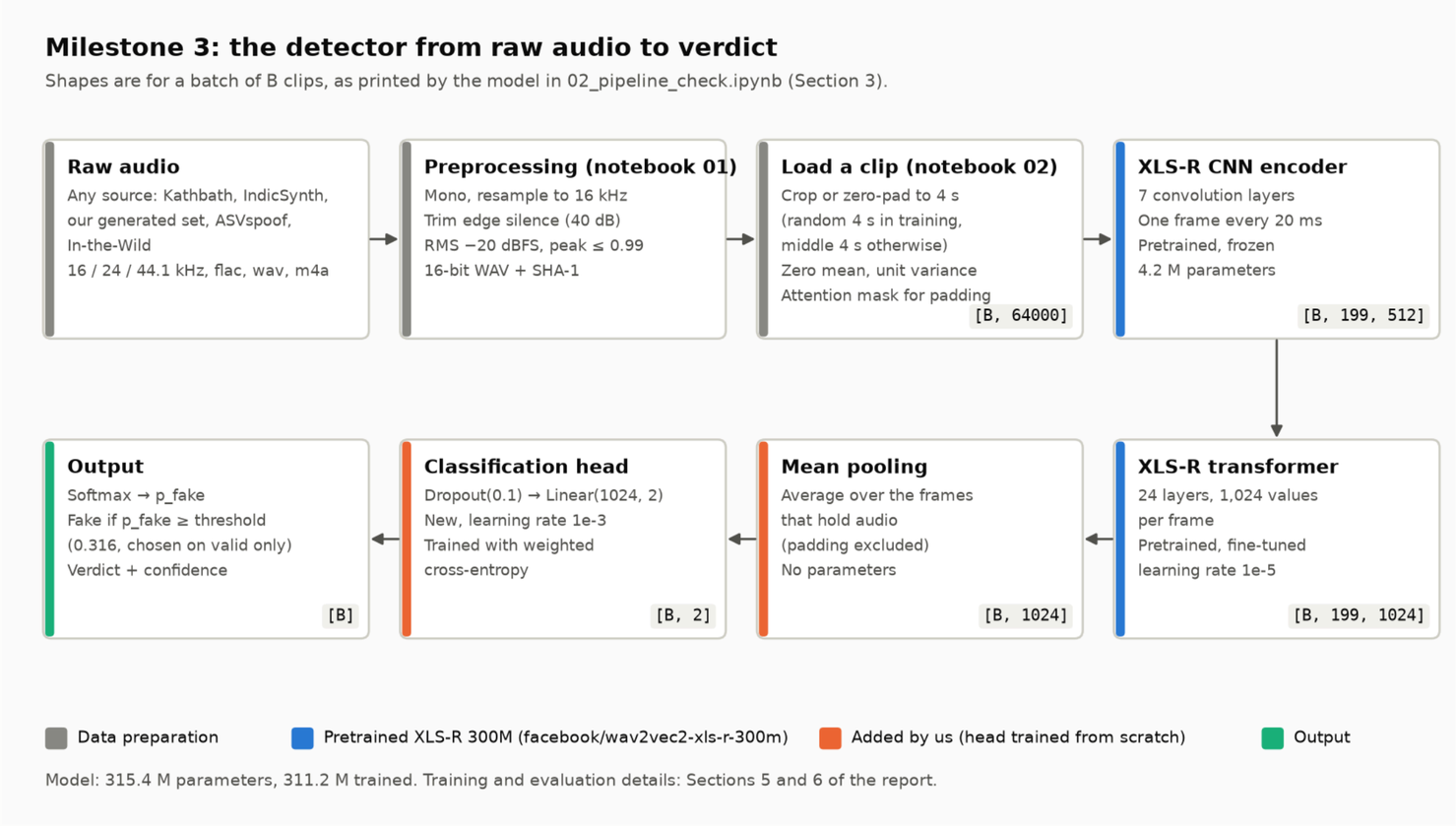
0
unseen-test speakers in train

0
unseen-test sentences in train

Things a model learns from the data use only train or valid: the class weights and the decision cut-off.

07 · THE MODEL

From raw audio to a verdict



08 · INSIDE THE MODEL

What goes in and what comes out

Stage	Shape	In plain words
Input	[B, 64000]	4 seconds of audio at 16 kHz, per clip
XLS-R CNN encoder (frozen)	[B, 199, 512]	One frame every 20 ms
XLS-R transformer (fine-tuned)	[B, 199, 1024]	Each frame, described with its context
Mean pooling	[B, 1024]	One summary per clip
Head: Dropout → Linear	[B, 2]	Two scores: real and fake
Softmax + cut-off	[B]	p_fake, then real or fake

315.4 M
parameters

311.2 M
trained

4 s
per clip; shorter ones padded

09 · WHY XLS-R

Why this model fits our problem

Knows our languages

Pretrained on 436,000 hours in 128 languages, including Hindi, Tamil and Telugu.

Handles new tools

Self-supervised models catch unseen attacks far better: 2.59% vs 29.12% EER in ASVspoof 5.

Fits a free GPU

One round over 1,492 clips took 3 minutes on one Kaggle T4.

We train on only two Indian voice tools, but must catch new ones. So we start from a model that already knows speech.

10 · OTHER OPTIONS

How it compares

Approach	Reads	Size	Best published EER	Knows other languages	Our use
LFCC-GMM	Hand-made features	Small	3.50–8.09%	No	Not used
LCNN	Spectrogram	Small CNN	1.92%	No	Baseline later
AASIST-L	Raw audio	~85K	0.99%	No	Baseline later
RawNet2	Raw audio	~25M	4.66%; 33.94% in the wild	No	Not used
XLS-R + AASIST	Raw audio	300M+	0.82%*	Yes	Option later
XLS-R + linear head (ours)	Raw audio	315M	Not yet measured	**Yes**	**Main model**

EER on ASVspoof 2019 LA, from our Milestone-1 review. *ASVspoof 2021 LA. The numbers come from different setups, so they aren’t a direct ranking.

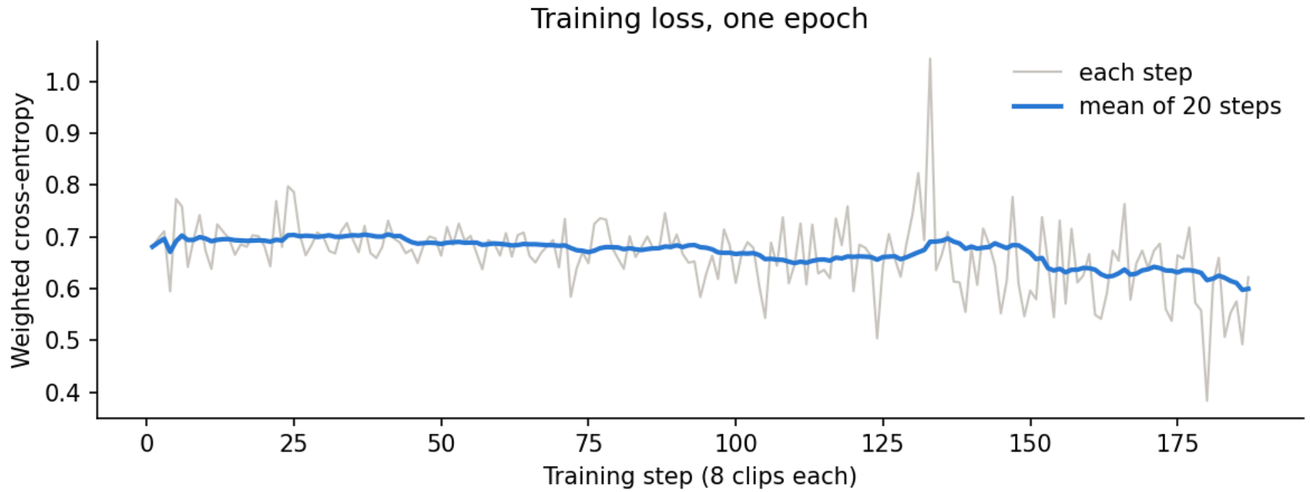
One round on a free GPU

1 epoch
187 steps of 8 clips

3.0 min
on one Kaggle T4

0.701 → 0.610
loss, start to end

1,526
clips scored in 174 s



The loss goes down, so the model is learning from the data.

Learning rates: 1e-5 for XLS-R, so it keeps what it knows; 1e-3 for the new head.

No augmentation in this run, on purpose.

How it learns, and how we judge it

Training loss

Cross-entropy: it pushes the fake score up for fakes and down for real clips. Classes are weighted by train counts; here both are 1.0.

Main metric: EER

The error rate where “fake passed as real” equals “real flagged as fake”. Lower is better. The standard in this field.

Also reported

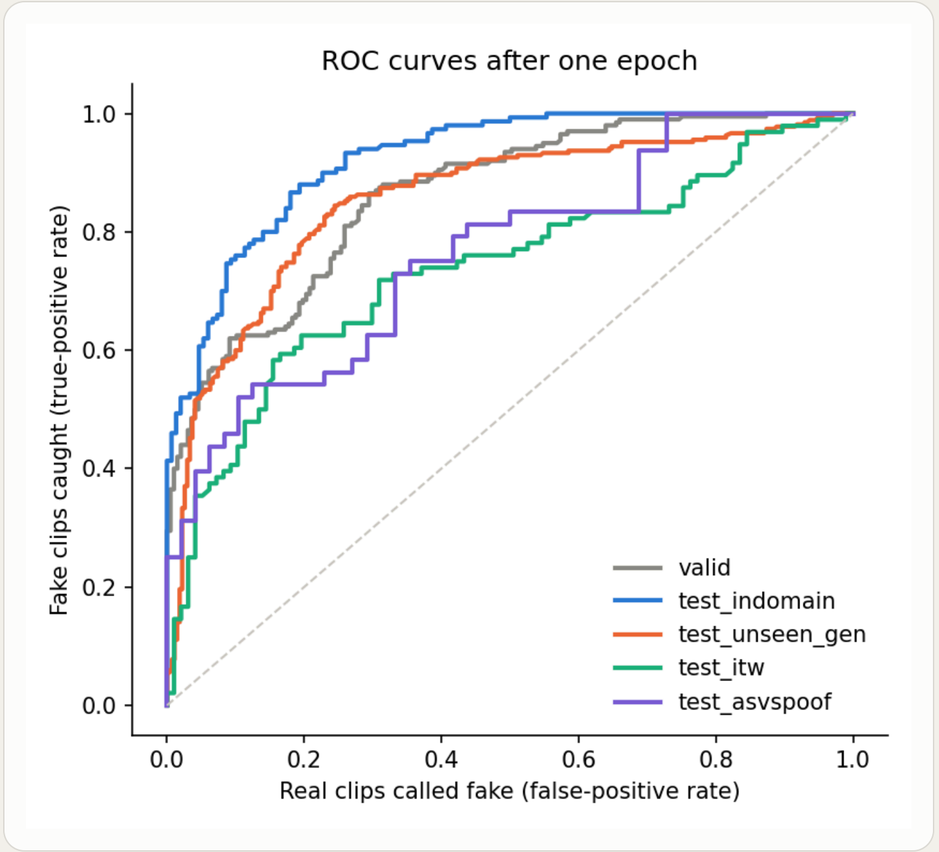
ROC AUC (how well scores rank fakes above real clips) and accuracy at one fixed cut-off.

The cut-off is 0.316: where the two errors are equal on valid. It’s fixed there and reused on every test.

Better than chance on every test

Test	EER	AUC
New speakers	17.7%	0.924
Unseen voice tools	20.6%	0.854
Real-world English	31.6%	0.735
Unseen English attacks	33.3%	0.763
valid	24.4%	0.863

After one round on 1,492 clips. Lower EER is better; AUC 0.5 is chance.



14 · SAMPLE OUTPUTS

What the model says, clip by clip

Clip	Test	Expected	Predicted	p_fake
Kathbath, Hindi	New speakers	real	real ✅	0.242
XTTS-v2, Telugu	New speakers	fake	real ❌	0.309
Kathbath, Telugu	Unseen tools	real	real ✅	0.264
MMS-TTS, Telugu	Unseen tools	fake	real ❌	0.283
In-the-Wild	Real-world	real	real ✅	0.266
In-the-Wild, unknown TTS	Real-world	fake	fake ✅	0.382
ASVspoof eval	English attacks	real	fake ❌	0.398
ASVspoof, attack A10	English attacks	fake	fake ✅	0.349

A clip is called fake when p_fake ≥ 0.316. One real and one fake clip per test, picked at random (seed 42).

Where the model struggles

26.7%

of IndicF5 fakes caught. It clones real voices, so it’s the hardest new tool

0.500

accuracy on ASVspoof eval: the cut-off from valid calls every clip fake

0.20–0.65

the range of all scores. They’re still bunched together after one round

Indic Parler-TTS is easy (90.0% caught) and MMS-TTS is in between (50.0%). **These are the things to watch in full training.**

English training data is now in

Milestone 2 feedback: **only the ASVspoof eval part had been explored.** We found a full copy with all three parts, and checked it against the official counts.

Part	Clips	Real	Fake	Attacks	Our split
Train	25,380	2,580	22,800	A01–A06	train
Dev	24,844	2,548	22,296	A01–A06	valid
Eval	71,237	7,355	63,882	A07–A19	test_asvspoof

292 English clips now train the model. The test uses attacks it has never seen.

What didn’t go to plan

What happened	What we did
Some Kathbath m4a files decoded to silence	Fixed: decode from a temporary file
The valid cut-off doesn’t carry over to ASVspoof eval	Study scores per source in full training
IndicF5 fakes are rarely caught	Track each voice tool separately
English train covers some attacks only (block sampling)	Use every attack in full training
Augmentation not used	Off on purpose for this check; on in Milestone 4
Only 4 s of each clip is read	Score several windows per clip
No explanation view yet	Add it in Milestone 4

What Milestone 4 does

Next step	Why
Train on the full split, more rounds, stop on valid	Reach the Milestone-1 targets
Turn on augmentation (noise, codec, phone, speed, pitch, echo)	Handle real calls and new tools
Train the LCNN and AASIST-L baselines	Show what XLS-R adds
Add cross-language and phone-channel tests	Complete the planned tests
Score longer clips in several windows	Use the whole clip, not just 4 s
Add the explanation view	Show which seconds drove each decision

The pipeline works end to end. Next, we make the model good.